

Preprocessing & Exceptions — Practical Syntax & Patterns (Python)

A compact, language-level guide for interviews and production. Covers validation, normalization, error taxonomy, try/except patterns, logging, resource cleanup, retries, and defensive programming. ASCII-only for portability.

1) Validation (LBYL) and Normalization

```
def ensure_sequence(name, seq, min_len=1):
    if seq is None:
        raise ValueError(f"{name} cannot be None")
    try:
        n = len(seq)
    except TypeError:
        raise TypeError(f"{name} must be a sized sequence")
    if n < min_len:
        raise ValueError(f"{name} length must be >= {min_len}, got {n}")

def ensure_numeric_iter(name, seq):
    import math
    for i, x in enumerate(seq):
        if not isinstance(x, (int, float)):
            raise TypeError(f"{name}[{i}] must be numeric, got {type(x).__name__}")
        if not math.isfinite(float(x)):
            raise ValueError(f"{name}[{i}] must be finite, got {x}")

def normalize_list_numbers(seq, clamp_min=None, clamp_max=None, round_ndigits=None):
    out = []
    for x in seq:
        y = float(x)
        if clamp_min is not None and y < clamp_min: y = clamp_min
        if clamp_max is not None and y > clamp_max: y = clamp_max
        if round_ndigits is not None: y = round(y, round_ndigits)
        out.append(y)
    return out

def dedupe_stable(seq):
    seen = set()
    res = []
    for x in seq:
        if x not in seen:
            seen.add(x)
            res.append(x)
    return res
```

2) Exceptions: Taxonomy and Usage

```
# Value domain / format
class ValidationError(ValueError): pass

# Postconditions / invariants
class PostconditionError(AssertionError): pass

# Resource / state errors
class ResourceError(RuntimeError): pass

def require(cond, msg="precondition failed"):
    if not cond:
        raise ValidationError(msg)

def ensure(cond, msg="postcondition failed"):
    if not cond:
        raise PostconditionError(msg)
```

3) try / except / else / finally and 'raise ... from ...'

```
def load_json(path):
    import json, os
    if not os.path.exists(path):
        raise FileNotFoundError(f"not found: {path}")
    f = None
    try:
        f = open(path, "r", encoding="utf-8")
        data = json.load(f)
    except json.JSONDecodeError as e:
        # chain for root cause
        raise ValidationError(f"invalid JSON: {path}") from e
```

```

except OSError as e:
    raise ResourceError(f"I/O error reading {path}") from e
else:
    # runs only if no exception
    return data
finally:
    if f is not None:
        try: f.close()
        except Exception: pass

```

4) Logging: levels and actionable messages

```

import logging
log = logging.getLogger(__name__)

def parse_int(s):
    try:
        return int(s)
    except (TypeError, ValueError) as e:
        log.warning("parse_int failed for %r: %s", s, e) # include input and reason
        raise

```

5) String parsing and robust conversion

```

def atoi32(s):
    s = (s or "").strip()
    if not s:
        raise ValueError("empty string")
    i, sign, n = 0, 1, len(s)
    if s[0] in "+-":
        sign = -1 if s[0] == "-" else 1
        i += 1
    val = 0
    INT_MIN, INT_MAX = -2**31, 2**31 - 1
    while i < n and s[i].isdigit():
        d = ord(s[i]) - 48
        if val > (INT_MAX - d) // 10:
            return INT_MAX if sign > 0 else INT_MIN
        val = val * 10 + d
        i += 1
    if i == (1 if s[0] in "+-" else 0):
        raise ValueError("no digits")
    return sign * val

```

6) File I/O and context managers

```

from contextlib import contextmanager

@contextmanager
def open_text(path):
    f = open(path, "r", encoding="utf-8")
    try:
        yield f
    finally:
        f.close()

def read_lines(path):
    try:
        with open_text(path) as f:
            return [ln.rstrip("\n") for ln in f]
    except FileNotFoundError:
        # handle or rethrow
        raise

```

7) Network/API retries with backoff (EAFP)

```

import time, random, urllib.error

def retry(func, retries=3, base=0.2, cap=2.0, jitter=True):
    delay = base
    for attempt in range(1, retries+1):
        try:
            return func()
        except (urllib.error.URLError, TimeoutError) as e:
            if attempt == retries:
                raise
            sleep = min(delay, cap)
            if jitter: sleep *= (0.5 + random.random()/2)
            delay = sleep * 2

```

```
time.sleep(sleep)
delay *= 2
```

8) Math guards: zero, overflow, epsilon comparisons

```
def safe_div(a, b, default=None):
    if b == 0:
        if default is not None: return default
        raise ZeroDivisionError("division by zero")
    return a / b

EPS = 1e-9
def feq(a, b): return abs(a-b) < EPS
def fle(a, b): return a < b or feq(a, b)
```

9) Optional: numpy/pandas preprocessing

```
def require_np_array(arr, ndim=None):
    import numpy as np
    if arr is None or not isinstance(arr, np.ndarray):
        raise TypeError("arr must be a numpy.ndarray")
    if ndim is not None and arr.ndim != ndim:
        raise ValueError(f"expected ndim={ndim}, got {arr.ndim}")
    if not np.all(np.isfinite(arr)):
        raise ValueError("array contains NaN/Inf")

def ensure_columns(df, required):
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise KeyError(f"missing columns: {missing}")
```

10) API skeleton: preconditions -> normalize -> compute -> postconditions

```
def api_compute(seq):
    # preconditions
    ensure_sequence("seq", seq, min_len=1)
    ensure_numeric_iter("seq", seq)

    # normalization
    xs = normalize_list_numbers(seq, clamp_min=0.0, round_ndigits=6)
    xs = dedupe_stable(xs) # example; depends on semantics

    # compute
    total = sum(xs)

    # postconditions
    if total < 0:
        raise PostconditionError("total must be non-negative")
    return total
```

11) Unit tests for error paths (pytest)

```
import pytest

def test_ensure_sequence_errors():
    with pytest.raises(ValueError):
        ensure_sequence("seq", [], min_len=1)
    with pytest.raises(TypeError):
        ensure_sequence("seq", 123, min_len=1)

def test_atoi32_overflow():
    assert atoi32("2147483648") == 2**31 - 1
    assert atoi32("-2147483649") == -2**31
```

12) Quick reference: choose an exception

# Value/format invalid	-> ValueError / ValidationError
# Wrong type	-> TypeError
# Missing index/key	-> IndexError / KeyError
# I/O / filesystem	-> OSError / FileNotFoundError
# Network timeouts	-> TimeoutError / URLError
# Zero division / overflow	-> ZeroDivisionError / OverflowError
# Invariant failure	-> AssertionError / PostconditionError
# Not implemented branch	-> NotImplementedError